

SnowPro Advanced: Data Scientist — Prep Guide

PJ's Academy · Prove you can do end-to-end ML on Snowflake.

The Data Scientist exam (DSA-C02) validates that you can run the full data-science lifecycle **inside Snowflake** — feature engineering, model training, deployment, and MLOps — using Snowpark, Snowflake ML, and Cortex.

Prerequisite: SnowPro Core certified.

Exam At A Glance

Item	Detail
Exam code	DSA-C02
Questions	~65
Duration	115 minutes
Passing score	~750/1000 scaled
Cost	\$375 USD
Prerequisite	SnowPro Core

Domain Weightings

#	Domain	Weight
1	Data Preparation & Feature Engineering	30%
2	Model Development	30%
3	Model Deployment, Ops & Monitoring	25%
4	Data Science Concepts & Snowflake Ecosystem	15%

Domain 1 — Data Preparation & Feature Engineering (30%)

Snowpark for feature engineering

- DataFrame API — lazy evaluation, transformations pushed down to Snowflake.
- Window functions and aggregations for features (rolling means, lags, ratios).
- Handling missing values (`fillna` , `dropna`), encoding categoricals.
- `snowflake.ml.modeling.preprocessing` — `StandardScaler` , `OneHotEncoder` , `OrdinalEncoder` , `MinMaxScaler` that run **distributed** in Snowflake.

Feature Store

- **Snowflake Feature Store** — define feature views, entities, and retrieve point-in-time-correct training sets.
- Avoiding **train/serve skew** and **data leakage** (no future data in features).

Sampling & splitting

- `TABLESAMPLE` , stratified sampling, reproducible splits with seeds.

```
from snowflake.ml.modeling.preprocessing import StandardScaler, OneHotEncoder
scaler = StandardScaler(input_cols=["income", "age"], output_cols=["income_s", "age_s"])
df_scaled = scaler.fit(train_df).transform(train_df)  # runs distributed in Snowflake
```



Domain 2 — Model Development (30%)

Snowflake ML modeling

- `snowflake.ml.modeling` mirrors scikit-learn / XGBoost / LightGBM APIs but trains on Snowflake compute.
- Supported: linear/logistic regression, random forest, XGBoost, LightGBM, K-Means, PCA, etc.
- Hyperparameter tuning with `GridSearchCV` / `RandomizedSearchCV` (distributed).

Cortex ML functions (SQL-native ML)

- `FORECAST` — time-series forecasting in one SQL function.
- `ANOMALY_DETECTION` — flag outliers.
- `CLASSIFICATION` — quick classification.
- `TOP_INSIGHTS` — automated driver analysis.

Cortex LLM functions

- `COMPLETE` , `SUMMARIZE` , `SENTIMENT` , `EXTRACT_ANSWER` , `TRANSLATE` , `EMBED_TEXT` .

```
-- Time-series forecast with zero ML code
CREATE SNOWFLAKE.ML.FORECAST sales_model(
  INPUT_DATA => TABLE(sales_history),
  TIMESTAMP_COLNAME => 'ds', TARGET_COLNAME => 'y');
CALL sales_model!FORECAST(FORECASTING_PERIODS => 30);
```

Evaluation

- Metrics via `snowflake.ml.modeling.metrics` — accuracy, ROC-AUC, RMSE, R^2 .
- Cross-validation, confusion matrices, calibration.



Domain 3 — Deployment, Ops & Monitoring (25%)

Model Registry

- `snowflake.ml.registry.Registry` — version models, log metrics, add metadata.
- Deploy a logged model as a **SQL-callable function** or Python function.

```
from snowflake.ml.registry import Registry
reg = Registry(session)
mv = reg.log_model(model, model_name="churn", version_name="v1",
                  metrics={"roc_auc": 0.91})
reg.get_model("churn").default.run(features_df) # inference
```

Inference patterns

- Batch scoring via UDFs; real-time via functions.
- Vectorized (batch) Python UDFs for throughput.

Monitoring

- **ML Observability** — track drift, performance decay over time.
- Logging predictions to tables; scheduled retraining with Tasks.

Domain 4 — DS Concepts & Ecosystem (15%)

- Bias/variance, overfitting, regularization, class imbalance strategies.
- Partner ecosystem: notebooks (Snowflake Notebooks), Streamlit, external ML tools.
- Data governance for ML — masking training data, lineage via ACCESS_HISTORY.

High-Yield Facts

- **Cortex ML functions** (FORECAST, ANOMALY_DETECTION, CLASSIFICATION) need **no Python** — pure SQL.
- `snowflake.ml.modeling` trains **inside** Snowflake — no data export.
- **Model Registry** versions models and exposes them as SQL functions.
- **Feature Store** gives point-in-time-correct training data (prevents leakage).
- **Cortex LLM** functions (COMPLETE, EMBED_TEXT) enable GenAI directly in SQL.
- **Vectorized UDFs** = pandas batches = faster inference.

Practice Questions (10)

- Q1.** Which runs a time-series forecast with no Python code? A) Snowpark ML B) SNOWFLAKE.ML.FORECAST C) UDF D) Task
- Q2.** Where do you version models and expose them as SQL functions? A) Stage B) Model Registry C) Stream D) Feature Store
- Q3.** The Feature Store primarily prevents: A) Slow queries B) Train/serve skew & leakage C) High cost D) Drift
- Q4.** `snowflake.ml.modeling.preprocessing.StandardScaler` runs: A) On your laptop B) Distributed inside Snowflake C) In Cortex D) In a UDF only
- Q5.** Which Cortex function returns text embeddings? A) COMPLETE B) EMBED_TEXT C) SENTIMENT D) SUMMARIZE
- Q6.** Fastest pattern for high-throughput Python inference: A) Row-by-row UDF B) Vectorized (batch) UDF C) External function D) Stored proc
- Q7.** Which detects outliers with a SQL function? A) ANOMALY_DETECTION B) FORECAST C) CLASSIFICATION D) TOP_INSIGHTS
- Q8.** To retrain a model on a schedule you use: A) Stream B) Task C) Pipe D) Share
- Q9.** ML Observability is used to monitor: A) Cost B) Drift & performance decay C) Security D) Storage
- Q10.** Which prevents future data leaking into training features? A) Masking B) Point-in-time-correct retrieval C) Cloning D) Sampling

✓ Answers

1-B · 2-B · 3-B · 4-B · 5-B · 6-B · 7-A · 8-B · 9-B · 10-B



3-Week Plan

- **Week 1:** Feature engineering (Snowpark + Feature Store).
 - **Week 2:** Model development (snowflake.ml + Cortex).
 - **Week 3:** Registry, deployment, monitoring + practice.
-

❄️ *Snowflake Mastery* — *PJ's Academy*

❄️ Snowflake Mastery — PJ's Academy · pjsacademy.com · hello@pjsacademy.com · @pjsacademy.datascience